How to run the code

Assignment 1: Reinforcement Learning · Vincent Yang · https://github.com/vi-yang-NU/RL

These instructions are for grading. Every answer, plot, evaluation statistic and validation result is in the report, and the repository includes the results the report was built from (`results/`), so **nothing has to be run to grade**. The steps below set up the code to rerun any part, and open a dashboard that shows each part's answers next to its plots and test evidence.

1. Quick start

You need Python 3.9 or newer on macOS or Linux (on Windows, use WSL). In a terminal:

```
git clone https://github.com/vi-yang-NU/RL.git
cd RL
./setup.sh --no-ssh           # creates .venv, installs the packages, checks the environment
source .venv/bin/activate
python run.py ui              # opens the dashboard in your browser at http://localhost:8501
```

`setup.sh` installs the packages in `requirements.txt` (NumPy, Matplotlib, PyTorch, Gymnasium with the Atari environments, ale-py) and `requirements-ui.txt` (Streamlit, pandas, reportlab) into a virtual environment in `.venv`; the first time takes a few minutes because of PyTorch. It ends with `python run.py check`, whose last line should read `ready`. `--no-ssh` skips the optional SSH setup for Northwestern's deepdish servers. Stop the dashboard with Ctrl+C in the terminal.

Without the script:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt -r requirements-ui.txt
python run.py check
```

2. Where to find each answer

The report and the dashboard follow the handout's order and use the same headings. In the dashboard, open the tab, then scroll to the section; in the report, the same section is under the part's heading.

Handout item	Dashboard tab › section	Rerun with
GitHub username, repository and link; these instructions	Overview	
2: MDP formulation and assumptions	2 · Value & Policy Iteration › MDP formulation, Assumptions	
2 (a) Enumeration, T = 500: optimal value function at time 0	2 · Value & Policy Iteration › (a) Enumeration, T = 500	`python run.py dp --method enumeration`
2 (b) Value iteration: optimal value function	2 · Value & Policy Iteration › (b) Value iteration, T = ∞	`python run.py dp --method value_iteration`

Handout item	Dashboard tab › section	Rerun with
2 (c) Policy iteration: optimal policy	2 · Value & Policy Iteration › (c) Policy iteration	`python run.py dp --method policy_iteration`
2: edge cases, validation tests, 3 more months	2 · Value & Policy Iteration › Edge cases, Tests for validating..., Additional validation...	`python run.py tests --section dp`
Part 1 (a) CartPole-v1: training curve, histogram, mean and standard deviation	3 · Policy Gradient Part 1 › (a) CartPole-v1	`python run.py pg --part 1 --env cartpole`
Part 1 (b) Pong-v5: the same	3 · Policy Gradient Part 1 › (b) Pong-v5	`python run.py pg --part 1 --env pong`
Part 1: edge cases, validation tests, 3 more months	3 · Policy Gradient Part 1 › Edge cases, Tests for validating..., Additional validation...	`python run.py tests --section pg1`
Part 2: the baseline chosen and how it is implemented	Policy Gradient Part 2 › Baseline: choice and implementation	
Part 2 (a) CartPole-v1 with the baseline	Policy Gradient Part 2 › (a) CartPole-v1 with baseline	`python run.py pg --part 2 --env cartpole`
Part 2 (b) Pong-v5 with the baseline	Policy Gradient Part 2 › (b) Pong-v5 with baseline	`python run.py pg --part 2 --env pong`
Part 2: edge cases, validation tests, 3 more months	Policy Gradient Part 2 › Edge cases, Tests for validating..., Additional validation...	`python run.py tests --section pg2`

The files behind them are in `results/` (section 8 lists the folders).

3. Using the dashboard

`python run.py ui` opens the dashboard at http://localhost:8501.

- **Tabs** along the top:
  - **Overview**: the submission details, a table of where each part is, progress, and the report and these instructions as PDFs to download.
  - **2 · Value & Policy Iteration**, **3 · Policy Gradient Part 1** and **Policy Gradient Part 2**: one tab per part of the handout.
  - **Setup (deepdish)**: one-time steps for running jobs on the deepdish GPU servers; not needed for grading.
- **A part's tab** lists the handout's questions in order, each in its own box: the question quoted from the handout, the required plots, for policy-gradient runs the mean and standard deviation over the 500 evaluation episodes, the validation tests, and the **Answer**. Click a validation test to expand it: what it does, how it validates the algorithm, each check (✓ or ✗), and its metrics, tables and figures.
- **The Run panel** at the top of each part's tab reruns that part; click its header to collapse it. Choose a job (for example *(a) CartPole-v1* or *Validation tests*), optionally add extra arguments, and press **Launch**. Under **Jobs**, choose a job to see its log and, while a

policy-gradient run trains, its reward curve (turn on auto-refresh to follow it). When the job has finished, press R to reload the page and its plots.

- **The sidebar** on the left sets where jobs run: **this computer** (the default) or a deepdish server, and the GPU. Hide it with « at its top; bring it back with » at the top left of the page. The deepdish connection settings in it are only needed to run on deepdish.

Rerunning a part overwrites its results in your copy of the repository; `git checkout results/` restores the submitted ones. For a policy-gradient run, a **Run tag** (or `--tag NAME` on the command line) writes to a separate folder instead and leaves the submitted run in place.

4. Rerunning from the command line

Every part runs through one entry point, `run.py`, from the repository root with the virtual environment active (`source .venv/bin/activate`); `python run.py -h` lists the commands.

Part	Command	Results in
Section 2 (a)–(c): enumeration, value and policy iteration	`python run.py dp`	`results/dp/`
Part 1 (a): CartPole-v1	`python run.py pg --part 1 --env cartpole`	`results/pg/part1_cartpole/`
Part 1 (b): Pong-v5	`python run.py pg --part 1 --env pong`	`results/pg/part1_pong/`
Part 2 (a): CartPole-v1 with a baseline	`python run.py pg --part 2 --env cartpole`	`results/pg/part2_cartpole/`
Part 2 (b): Pong-v5 with a baseline	`python run.py pg --part 2 --env pong`	`results/pg/part2_pong/`

Section 2. `python run.py dp` solves the shuttle-dispatch MDP (K = 15, c_f = 100, c_h = 2, A_t ~ Unif{1, ..., 5}, at most 200 customers waiting, γ = 0.95) with all three methods; `--method enumeration`, `--method value_iteration` or `--method policy_iteration` runs one of them. Each method saves its solution as JSON and its plot: `enumeration_value.png`, `value_iteration_value.png` and `policy_iteration_policy.png`.

Policy gradient. Each run trains the policy network with Adam, then rolls out 500 episodes with the trained policy. CartPole-v1 trains on a CPU; Pong-v5 is much slower and is best run on a GPU (section 6). A run's folder contains:

- `training_curve.png`: episode reward against training episode, with the 100-episode moving average;
- `eval_histogram.png`: histogram of the 500 evaluation rewards;
- `eval.json`: the 500 evaluation rewards, their mean and their standard deviation;
- `config.json` (the settings used) and `train.json`.

The defaults follow the handout: learning rate 0.01, discount factor 0.95 for CartPole and 0.99 for Pong, and 500 evaluation episodes. Options:

Option	Effect
`--episodes N`	number of training episodes (default 1000 for CartPole, 2000 for Pong)
`--lr X`, `--gamma X`	Adam learning rate and discount factor
`--baseline NAME`	Part 2 baseline (default: `DEFAULT_BASELINE` in `rl/pg/baselines.py`)
`--seed N`	random seed (default 0)
`--device D`	`auto` (the GPU when one is visible), `cpu` or `cuda`
`--eval-episodes N`	number of evaluation episodes (default 500)
`--tag NAME`	save to `results/pg/<run>_NAME/`, next to the submitted run
`--eval-only`	skip training and evaluate the run's saved `policy.pt`
`--set KEY=VALUE`	extra setting passed to the training code (repeatable)

5. Validation experiments

The experiments that check that each algorithm behaves as predicted are in `tests/test_dp.py`, `tests/test_pg_part1.py` and `tests/test_pg_part2.py`. Each one states what it does and how it validates the algorithm; the report's "Tests for validating the correctness of your algorithm" sections list them with their results.

```
python run.py tests --list                # every experiment, by section
python run.py tests --section dp          # dp, pg1, pg2 or all
python run.py tests --section pg1 -k TEXT # only experiments whose name or title contains TEXT
```

Each experiment ends as **passed** (all its checks hold), **failed** (a check does not hold), **error** (it raised an exception) or **recorded** (evidence without pass/fail checks). Results go to `results/tests/<section>.json`, with figures in `results/tests/<section>/`.

6. Running on deepdish (one GPU card)

The handout requires a single GPU card on deepdish. Clone the repository there, run `./setup.sh --server` (it detects the servers' GTX 1080 cards and installs a PyTorch build that supports them), check which cards are free with `nvidia-smi`, and choose one with `CUDA_VISIBLE_DEVICES`:

```
ssh <netid>@mlds-deepdish3.ads.northwestern.edu       # campus network or the VPN
cd RL && source .venv/bin/activate
CUDA_VISIBLE_DEVICES=0 python run.py pg --part 1 --env pong
```

For long runs, `scripts/jobs.sh` starts the command in the background on one card, so it keeps running after you log out:

```
# arguments: job name, GPU index, command
scripts/jobs.sh start part1_pong 0 python run.py pg --part 1 --env pong
scripts/jobs.sh status                    # state, exit code and GPU of every job
scripts/jobs.sh tail part1_pong 50        # last 50 lines of the job's log
scripts/jobs.sh stop part1_pong
```

7. Rebuilding the report

```
python run.py report
```

writes `report/report.md` and `report/report.pdf` (the report) and `report/instructions.pdf` (these instructions) from the saved answers, plots, evaluation statistics and test results. The **Overview** tab of the dashboard does the same.

8. Repository layout

Path	Contents
`run.py`	entry point for every command above
`rl/dp/`	Section 2: the shuttle-dispatch MDP, enumeration, value iteration and policy iteration
`rl/pg/`	policy gradient: environments, policy network, training and evaluation, baselines
`rl/plotting.py`	the plots the handout requires
`tests/`	the validation experiments and the harness that runs them
`results/dp/`, `results/pg/<run>/`, `results/tests/`	Section 2 results, one folder per policy-gradient run, validation results
`report/`	the report (`report.pdf`, `report.md`), the saved answers, these instructions and the report builder
`ui/`	the dashboard
`scripts/jobs.sh`	background job runner, one GPU card per job